

GCB6206 Homework 5: Offline RL

Jae Yong Lee
2025961215

1 Introduction

In this assignment, you will implement one of the fundamental algorithms in offline reinforcement learning: Conservative Q-Learning (CQL). You will be experimenting with different hyperparameters and offline datasets that have been provided to you. In this assignment, we provide a random policy and an expert policy for you to compare the performance according to the quality of the collected data.

Your objectives are as follows:

1. Implement and train an offline reinforcement learning algorithm, CQL
2. Experiment with the key hyperparameters and explore how they affect performance of your RL agent
3. Analyze differences between the quality of different offline datasets, Random and Expert

Writing the report: Provide your responses for questions with %TODO tags in this LaTeX template. Submit all the requested values in tables, and put in all requested plots/images. You should also include all your reasoning and text responses in the PDF.

2 Codebase

In this assignment, you mainly work with the following codes:

- `gcb6206/scripts/run_cql.py`: the main training loop for your CQL implementation.
- `gcb6206/critics/cql_critic.py`: the CQL learner you will implement.

For each problem, we specify which files you need to complete. Sections that need to be filled have been marked with TODO tags.

You can install dependencies with the following script:

```
conda activate [your_env]
pip install -r requirements.txt
pip install -e .
```

3 Tips for experiments

In this homework, our code is computationally light; if you use a RTX 3090 GPU, the GPU utilization will be up to 20%. To utilize the full power of GPUs, we recommend you to run multiple jobs simultaneously. You can do it by (1) executing multiple bash terminals and execute the training script for each bash terminal; (2) using “&” command to run the jobs in background; or (3) using `tmux`. We recommend trying `tmux` since it is one of the powerful utility tools that you can use inside linux servers. One of the advantages of `tmux` is

that your work status would be saved inside a tmux session, so it will keep running even if you loose the ssh connection.

4 Preliminaries

Environments: In this assignment, you will be working with the Pointmass environment. The goal for the Pointmass environment is to navigate a grid world of varying difficulties: **easy**, **medium**, and **hard** to reach the ‘goal’ location. Sample environments for each difficulty have been visualized in Figure 1. The observation consists of (y, x) -location of the point mass and the action space is 5 discrete numbers indicating **NO MOVE**, **LEFT**, **RIGHT**, **UP**, and **DOWN**. The agent receives 1 reward and terminates when the agent reaches the goal, otherwise receives reward of 0.

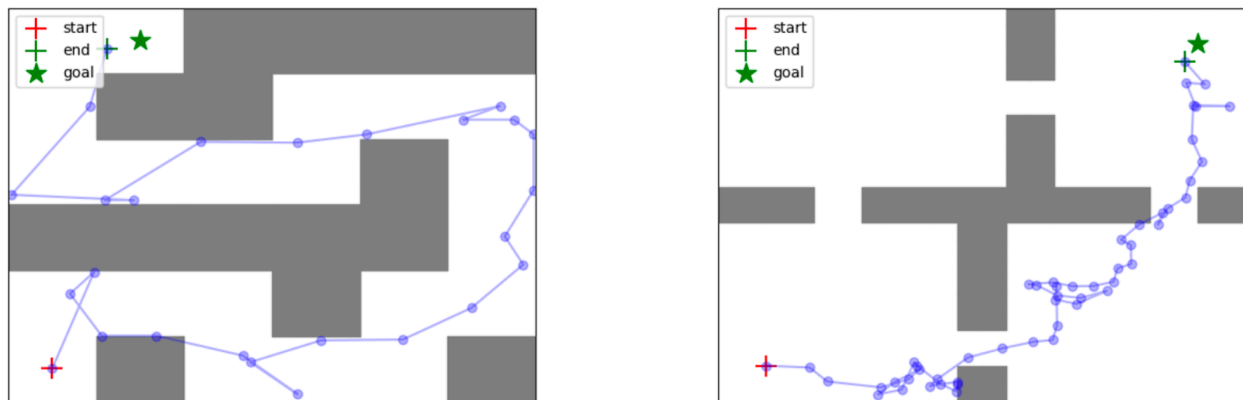


Figure 1: Visualization of the Pointmass environments with **Medium** (left) and **Hard** (right) difficulties for navigation and reaching the goal.

Offline datasets: For offline RL, we have provided you with a set of data collection strategies which are used to collect the offline dataset. These trajectories then serve as the training dataset for your offline RL algorithms. The offline transitions dataset $\mathcal{D} = \{(s_i, a_i, r_i, s_{i+1})_i\}$ consists of tuples of the current state, action, reward, and future state. In this assignment, you will experiment with two datasets generated by (1) random policy (100000 steps) and (2) expert policy (20000 steps).

5 Conservative Q-Learning

5.1 Implement CQL

In this problem, you’ll be implementing the **Conservative Q-Learning** (CQL) algorithm (a DQN version for the discrete environment). The goal of CQL is to prevent overestimation of the policy value. The overall CQL objective is given by the standard TD error objective augmented with the CQL regularizer weighted by α :

$$L_Q(\theta) = \frac{1}{N} \sum_{i=1}^N \left[\left(Q(s_i, a_i) - \left(r(s_i, a_i) + \gamma \max_{a'} [Q(s'_i, a')] \right) \right)^2 \right] + \alpha \frac{1}{N} \sum_{i=1}^N \left[\log \left(\sum_a \exp(Q(s_i, a)) \right) - Q(s_i, a_i) \right]. \quad (1)$$

Fill in the TODOs in the following file:

- gcb6206/critics/cql_critic.py

Once you’ve filled in all of the TODO commands, you should be able to run CQL on the expert dataset with the following command:

```
python gcb6206/scripts/run_cql.py --env_name PointmassMedium-v0 \
  --exp_name cql_alpha_{$ALPHA}_expert \
  --dataset_name expert --cql_alpha={$ALPHA}
```

Run CQL with $\alpha = \{0.0, 0.1\}$ in **expert** dataset. Please report the **Eval_AverageReturn**, **Eval_StdReturn** of the last step in your tensorboard log file in the table. Your CQL implementation should achieve about at least 0.9 in **expert** dataset with $\alpha = 0.1$ in **Medium** difficulty. The code will run for 50000 iterations and takes around 30 minutes. Also, attach the **eval_last_traj.png** image, which visualizes your agent’s last trajectory, and the overestimation value curve, $\frac{1}{N} \sum_{i=1}^N [\log (\sum_a \exp (Q(s_i, a))) - Q(s_i, a_i)]$.

What does $\alpha = 0.0$ signify in terms of the algorithm (hint: think about an algorithm on top of which we developed CQL)? Explain the difference in the performance gap and the overestimation value with respect to α .

Answer:

- **Significance of $\alpha = 0.0$:** When $\alpha = 0.0$, the CQL regularization term in the objective function vanishes. This signifies that the algorithm reverts to standard Deep Q-Learning (DQN). We are essentially training a standard DQN agent on a fixed offline dataset without any conservative constraints.
- **Performance Gap:** There is a significant performance gap. With $\alpha = 0.0$ (DQN), the agent performs poorly because standard DQN in an offline setting tends to overestimate the values of out-of-distribution (OOD) actions, leading to a failed policy. With $\alpha = 0.1$ (CQL), the agent achieves high returns (at least 0.9) because the conservative regularizer suppresses the Q-values of OOD actions, forcing the agent to stay within the support of the expert dataset.
- **Overestimation Value:** With $\alpha = 0.0$, the overestimation value ($\log \sum \exp(Q) - Q_{data}$) is high (positive). This confirms that without regularization, the Q-function assigns improperly high values to unseen actions. With $\alpha = 0.1$, the overestimation value is low (or negative), as the CQL loss explicitly minimizes this term to prevent overestimation.

Table 1: Average Performance with different α in expert dataset.

	Mean	Standard Deviation
$\alpha = 0.0$	0.0	0.0
$\alpha = 0.1$	1.0	0.0

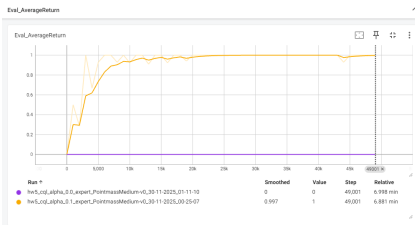


Figure 2: Evaluation Average Return on Expert Dataset (Comparison of $\alpha = 0.0$ and $\alpha = 0.1$).

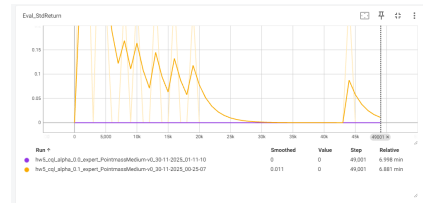


Figure 3: Evaluation Standard Deviation on Expert Dataset (Comparison of $\alpha = 0.0$ and $\alpha = 0.1$).

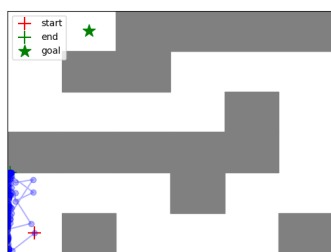


Figure 4: Trajectory plot for Expert dataset with $\alpha = 0.0$.

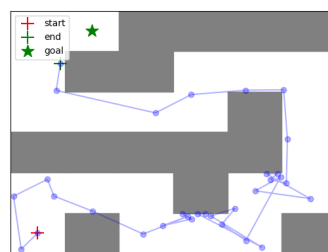


Figure 5: Trajectory plot for Expert dataset with $\alpha = 0.1$.

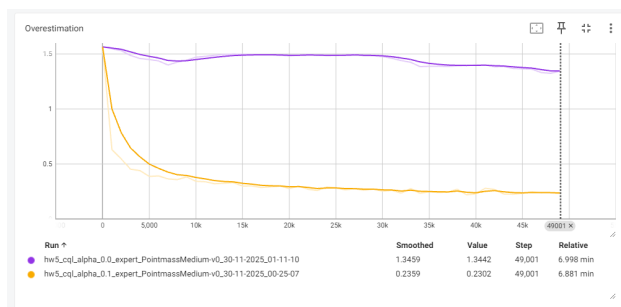


Figure 6: Overestimation of Q-values during training on Expert dataset (Comparison of $\alpha = 0.0$ and $\alpha = 0.1$).

5.2 Comparison between Expert dataset and Random dataset (5 points)

Compare the learned policies for the expert dataset and random dataset on the `PointmassMedium-v0` environment with $\alpha = \{0.0, 0.1\}$ by reporting the eval average return and standard deviation. Execute the below command to execute CQL with random dataset:

```
python gcb6206/scripts/run_cql.py --env_name PointmassMedium-v0 \
  --exp_name cql_alpha_{$ALPHA}_random \
  --dataset_name random --cql_alpha={$ALPHA}
```

Attach the `eval_last_traj.png` image denoting your agent's last trajectory for each run.

You may have noticed that the performance gap between $\alpha = 0.0$ and $\alpha = 0.1$ is not as large in the random dataset as it was in the previous experiment with the expert dataset. Explain why $\alpha = 0.0$ performs well on the random dataset but fails on the expert dataset. Hint: CQL aims to prevent overestimation of 'unseen' actions.

Answer: The primary reason lies in the coverage of the dataset (data distribution).

- **Expert Dataset:** This dataset contains only optimal trajectories, covering a very narrow part of the state-action space. Consequently, there are many "unseen" (out-of-distribution) actions. Standard DQN ($\alpha = 0.0$) tends to overestimate the Q-values of these unseen actions, causing the agent to deviate from the optimal path and fail.

- **Random Dataset:** This dataset is collected by a random policy, which explores the state-action space much more broadly. Because the dataset covers most possible transitions (including suboptimal ones), the Q-function can learn correct values for most actions solely from the data. Since there are fewer “unseen” actions to overestimate, the conservative penalty (α) is not strictly necessary, allowing the standard DQN ($\alpha = 0.0$) to perform well.

Table 2: Average Performance with different datasets and α .

	Mean	Standard Deviation
Random; $\alpha = 0.0$	1.0	0.0
Random; $\alpha = 0.1$	1.0	0.0
Expert; $\alpha = 0.0$	0.0	0.0
Expert; $\alpha = 0.1$	1.0	0.0

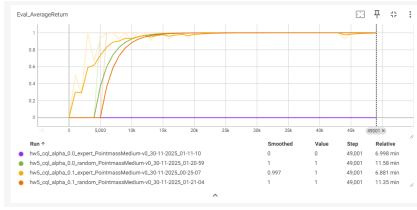


Figure 7: Evaluation Average Return on Expert and Random dataset.

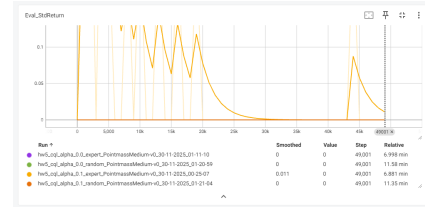


Figure 8: Evaluation Standard Deviation on Expert and Random dataset.

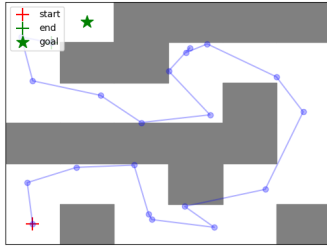


Figure 9: Trajectory plot for Random dataset with $\alpha = 0.0$.

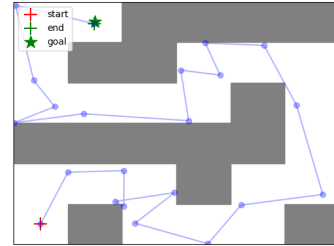


Figure 10: Trajectory plot for Random dataset with $\alpha = 0.1$.

6 Submission

Please submit the “report” hw5_[YourStudentID].pdf.